Gaussian Mixture Models: Tutorial (Expanded)

AI 500 - Programming Assignment 2

Fall 2025

1 Introduction and Motivation

1.1 Why Mixture Models?

Real-world data often arises from multiple underlying populations or processes. A mixture model explicitly represents this idea: the observed data is produced by a combination of a small number of simpler distributions (components), each representing a latent subpopulation or generative mechanism.

Examples and why a single Gaussian is insufficient:

- **Biometric data (heights)**: Heights often show two peaks—one for adult males, another for adult females. A single Gaussian would place one bell curve over the entire range and thus either underfit the tails or misplace the center. A mixture of two Gaussians can place two bell curves near the two modes and capture each subpopulation's spread.
- **Image segmentation (pixel intensities)**: An image that contains a foreground object and background may show two distinct intensity clusters. Using two Gaussian components lets us model each component's intensity distribution and then classify pixels probabilistically.
- **Customer behavior (purchase amounts)**: A retailer may have casual buyers (small purchases) and premium buyers (large purchases). Mixtures let us model the two spending patterns and infer the proportion of each customer type.

Key conceptual points:

- A **probability distribution** (like a Gaussian) describes how likely different values of a variable are. If data are generated from several processes, each process has its own distribution; a mixture simply sums those distributions weighted by how common each process is.
- The **challenge** is that we observe the combined data, not which process produced each observation. So we must estimate both component distributions and how likely each observation came from each component.

Key Insight: Because we do not observe which component generated each sample, our estimation problem is one with **latent (hidden) variables**. The Expectation–Maximization (EM) algorithm is an efficient, general-purpose method to handle such latent-variable maximum-likelihood problems by alternating between estimating the latent variables (soft

assignments) and updating the parameters.

2 Mathematical Foundations

2.1 The Gaussian Distribution — notation and intuition

A univariate Gaussian (normal) distribution with mean μ and variance σ^2 has probability density function:

$$\mathcal{N}(x \mid \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right). \tag{1}$$

What each piece means:

- μ is the **center** or expected value: it is the point around which samples are likely to cluster.
- σ^2 is the **variance**, measuring how spread-out the distribution is. Larger variance means the distribution is wider.
- The normalization factor $\frac{1}{\sqrt{2\pi\sigma^2}}$ ensures the density integrates to 1 (total probability mass is 1).
- The exponential term $\exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$ declines rapidly as $|x-\mu|$ increases; the quadratic in the exponent penalizes large deviations.

Density vs. probability (simplified:
The function $\mathcal{N}(x \mid \mu, \sigma^2)$ gives a **density**, not a direct probability for a single point when x is continuous. To get the probability that x is within an interval, you integrate the density over that interval.

MLE for a Gaussian (derivation sketch):
Given independent samples $x_1, \ldots, x_n$ from $\mathcal{N}(\mu, \sigma^2)$, the log-likelihood is:

$$\ell(\mu, \sigma^2) = \sum_{i=1}^{n} \log \mathcal{N}(x_i \mid \mu, \sigma^2) = -\frac{n}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(x_i-\mu)^2. \tag{2}$$

Setting derivatives w.r.t. μ and σ^2 to zero yields:

$$\hat{\mu}_{\text{MLE}} = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \hat{\sigma}^2_{\text{MLE}} = \frac{1}{n}\sum_{i=1}^{n}(x_i-\hat{\mu})^2. \tag{3}$$

Intuition: maximize the likelihood by centering the Gaussian at the sample mean and setting variance equal to the sample variance.

2.2 Mixture Models — formal definition and intuition

A finite mixture model with K components produces data according to:

$$p(x \mid \Theta) = \sum_{k=1}^{K} \pi_k \, p_k(x \mid \theta_k), \tag{4}$$

where:

- π_k is the **mixing weight** for component k. It represents the prior probability that an observation comes from component k. The weights satisfy $\pi_k \geq 0$ and $\sum_{k=1}^{K} \pi_k = 1$.

- $p_k(x \mid \theta_k)$ is the density of component k parameterized by θ_k (for Gaussians, $\theta_k = (\mu_k, \sigma_k^2)$).
- Θ denotes the full set of parameters: $\Theta = \{\pi_1, \ldots, \pi_K, \theta_1, \ldots, \theta_K\}$.

Generative story (intuitive procedure):

1. Draw component index $z \sim \text{Categorical}(\pi_1, \ldots, \pi_K)$. That means $P(z = k) = \pi_k$.
2. Given $z = k$, draw x from the density $p_k(\cdot \mid \theta_k)$.

This story clarifies the **latent variable** z: we do not observe z when we see x, so z is hidden.

Why mixture? Because the observed data distribution is a **weighted average** of component distributions. If each component is simple (Gaussian), the mixture can represent more complex, multimodal shapes.

2.3 Gaussian Mixture Model (GMM) — parameters and explicit form

A GMM uses Gaussian component densities:

$$p(x \mid \Theta) = \sum_{k=1}^{K} \pi_k \mathcal{N}(x \mid \mu_k, \sigma_k^2). \tag{5}$$

For 1-dimensional data the explicit expanded form is:

$$p(x \mid \Theta) = \sum_{k=1}^{K} \pi_k \cdot \frac{1}{\sqrt{2\pi\sigma_k^2}} \exp\left(-\frac{(x-\mu_k)^2}{2\sigma_k^2}\right). \tag{6}$$

Parameter set: $\Theta = \{\pi_1, \ldots, \pi_K, \mu_1, \ldots, \mu_K, \sigma_1^2, \ldots, \sigma_K^2\}$.
Special case ($K = 2$):

$$p(x \mid \Theta) = \pi_1 \mathcal{N}(x \mid \mu_1, \sigma_1^2) + (1 - \pi_1)\mathcal{N}(x \mid \mu_2, \sigma_2^2). \tag{7}$$

Intuition (simplified):

- π_k describes how **common** each mode is.
- μ_k describes where each mode is centered.
- σ_k^2 describes how wide each mode is.

3 The Parameter Estimation Problem

3.1 The challenge — likelihood and why it is hard

Goal: Given observed data $\mathcal{D} = \{x_1, \ldots, x_n\}$, choose parameters Θ that maximize the likelihood $p(\mathcal{D} \mid \Theta)$.

Because the data points are independent (i.i.d.), the log-likelihood is:

$$\ell(\Theta) = \log p(\mathcal{D} \mid \Theta) = \sum_{i=1}^{n} \log p(x_i \mid \Theta) \tag{8}$$

$$= \sum_{i=1}^{n} \log \left(\sum_{k=1}^{K} \pi_k \mathcal{N}(x_i \mid \mu_k, \sigma_k^2) \right). \tag{9}$$

Why direct maximization is hard:

- The log of a sum, $\log\left(\sum_k \pi_k \mathcal{N}(\cdot)\right)$, does not simplify into a sum of logs. Therefore, the derivatives of $\ell(\Theta)$ couple parameters across components in a way that does not yield closed-form solutions for μ_k, σ_k^2, or π_k simultaneously.
- Intuitively, we would like to treat each component separately (as we do for a single Gaussian), but the data assignment to components is unknown.

3.2 Latent variable formulation — complete vs incomplete data

Introduce latent (hidden) indicator variables $z_i \in \{1, \ldots, K\}$ where $z_i = k$ indicates that x_i was generated by component k.

- **Complete data**: $\{(x_i, z_i)\}_{i=1}^n$. If we had z_i we could maximize the complete-data likelihood by treating each component independently.
- **Incomplete data**: $\{x_i\}_{i=1}^n$. This is what we actually observe.

Joint (complete-data) distribution:

$$p(x_i, z_i = k \mid \Theta) = \pi_k \, \mathcal{N}(x_i \mid \mu_k, \sigma_k^2). \tag{10}$$

Marginal (observed-data) distribution:

$$p(x_i \mid \Theta) = \sum_{k=1}^{K} p(x_i, z_i = k \mid \Theta) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(x_i \mid \mu_k, \sigma_k^2). \tag{11}$$

Key observation: If we *knew* the z_i values, parameter estimation would reduce to estimating means and variances within each group of points assigned to the same z. The unknown z_i create a circular dependency: to infer z_i we need Θ; to infer Θ we need z_i. EM resolves this by iterating between estimating the expected z_i values (E-step) and maximizing parameters given those expectations (M-step).

4 Expectation–Maximization (EM) Algorithm

4.1 Algorithm overview — high level intuition

EM alternates two steps:

1. **E-step (Expectation)**: Given the current parameter estimate $\Theta^{(t)}$, compute the posterior distribution of the latent variables $p(z_i \mid x_i, \Theta^{(t)})$. These posteriors act as soft assignments of each data point to components.
2. **M-step (Maximization)**: Treat the posteriors as weights and maximize the expected complete-data log-likelihood with respect to Θ. This yields closed-form parameter updates for GMMs.

Intuitively:

- The E-step answers: "Given our current model, how likely is it that this data point came from each component?"
- The M-step answers: "Given these soft assignments, what parameter values best explain the data?"

Guarantee: EM increases (or leaves unchanged) the observed-data log-likelihood on every iteration; it converges to a stationary point (which can be a local maximum or saddle point).

4.2 Responsibilities (posterior probabilities)

Define the **responsibility** γ_{ik} as the posterior probability that component k generated x_i:

$$\gamma_{ik} \equiv p(z_i = k \mid x_i, \Theta) = \frac{p(z_i = k, x_i \mid \Theta)}{p(x_i \mid \Theta)}. \tag{12}$$

Using the joint and marginal formulas:

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(x_i \mid \mu_k, \sigma_k^2)}{\sum_{j=1}^{K} \pi_j \mathcal{N}(x_i \mid \mu_j, \sigma_j^2)}. \tag{13}$$

Interpretation and properties:

- $\gamma_{ik} \in [0, 1]$.
- For each data point i, $\sum_{k=1}^{K} \gamma_{ik} = 1$.
- In simpler words: γ_{ik} is the **posterior probability** (probability after seeing the data) that the latent label z_i equals k. It mixes prior belief π_k (how common component k is before seeing the point) with how well component k explains x_i (the likelihood $\mathcal{N}(x_i \mid \mu_k, \sigma_k^2)$).

Bayes' theorem intuition:

- Posterior $\propto$ Prior $\times$ Likelihood.
- Here: $p(z = k \mid x) \propto \pi_k \cdot p(x \mid z = k)$.

4.3 E-step — computing responsibilities numerically

Given the parameters $\Theta^{(t)}$, compute:

$$\gamma_{ik}^{(t)} = \frac{\pi_k^{(t)} \mathcal{N}(x_i \mid \mu_k^{(t)}, \sigma_k^{2(t)})}{\sum_{j=1}^{K} \pi_j^{(t)} \mathcal{N}(x_i \mid \mu_j^{(t)}, \sigma_j^{2(t)})}. \tag{14}$$

Numerical stability (practical remark):
The Gaussian densities can be extremely small for distant points; computing them directly may underflow to zero in floating point arithmetic. To avoid this, work in log-space using the **log-sum-exp trick**:

- Compute $\log\left(\pi_k \mathcal{N}(x_i | \mu_k, \sigma_k^2)\right)$ for each k, subtract the maximum across k before exponentiating, and normalise. This avoids exponentiating very negative numbers.
- Implementation-level tip: Use `scipy.special.logsumexp` or equivalent.

4.4 M-step — deriving the parameter updates

We maximize the **expected complete-data log-likelihood** given the current responsibilities. Define:

$$Q(\Theta \mid \Theta^{(t)}) = \mathbb{E}_{Z|X,\Theta^{(t)}}\left[\log p(X, Z \mid \Theta)\right] \tag{15}$$

$$= \sum_{i=1}^{n} \sum_{k=1}^{K} \gamma_{ik}^{(t)} \log\left(\pi_k \mathcal{N}(x_i \mid \mu_k, \sigma_k^2)\right). \tag{16}$$

The M-step maximizes $Q(\Theta \mid \Theta^{(t)})$ with respect to Θ. Because Q decomposes by component k, we can update parameters for each k independently.

4.4.1 Effective counts

Define the effective (fractional) sample size of component k:

$$N_k \equiv \sum_{i=1}^{n} \gamma_{ik}^{(t)}. \tag{17}$$

This is the expected number of points assigned to component k under the current posterior.

4.4.2 Update mixing weights π_k (derivation with Lagrange multiplier)

We want to choose $\{\pi_k\}$ to maximize

$$\sum_{k=1}^{K} N_k \log \pi_k \quad \text{subject to} \quad \sum_{k=1}^{K} \pi_k = 1. \tag{18}$$

Use a Lagrangian $\mathcal{L}(\pi, \lambda) = \sum_k N_k \log \pi_k + \lambda\big(1 - \sum_k \pi_k\big)$. Take derivative w.r.t. π_k and set to zero:

$$\frac{\partial \mathcal{L}}{\partial \pi_k} = \frac{N_k}{\pi_k} - \lambda = 0 \quad \Rightarrow \quad \pi_k = \frac{N_k}{\lambda}. \tag{19}$$

Using the constraint $\sum_k \pi_k = 1$ gives $\lambda = \sum_k N_k = n$. Hence:

$$\boxed{\pi_k^{(t+1)} = \frac{N_k}{n}} \tag{20}$$

Interpretation: set the mixing weight to the fraction of the total (expected) data points explained by component k.

4.4.3 Update means μ_k

Maximize Q w.r.t. μ_k: terms involving μ_k are

$$\sum_{i=1}^{n} \gamma_{ik} \left(-\frac{1}{2\sigma_k^2} (x_i - \mu_k)^2 \right) \tag{21}$$

(plus constants w.r.t. μ_k). Differentiate w.r.t. μ_k and set to zero:

$$\frac{\partial}{\partial \mu_k} \left(-\frac{1}{2\sigma_k^2} \sum_i \gamma_{ik} (x_i - \mu_k)^2 \right) = -\frac{1}{2\sigma_k^2} \cdot 2 \sum_i \gamma_{ik} (\mu_k - x_i) = 0. \tag{22}$$

This yields:

$$\sum_i \gamma_{ik} (\mu_k - x_i) = 0 \quad \Rightarrow \quad \mu_k^{(t+1)} = \frac{\sum_{i=1}^{n} \gamma_{ik}^{(t)} x_i}{N_k}. \tag{23}$$

Interpretation: a weighted mean where each data point contributes proportionally to the posterior probability that it belongs to component k.

4.4.4 Update variances σ_k^2

Maximize Q w.r.t. σ_k^2. Extract the relevant terms:

$$\sum_{i=1}^{n} \gamma_{ik} \left(-\frac{1}{2} \log \sigma_k^2 - \frac{(x_i - \mu_k)^2}{2\sigma_k^2} \right). \tag{24}$$

Differentiate w.r.t. σ_k^2 and set to zero. The derivative yields:

$$-\frac{N_k}{2\sigma_k^2} + \frac{1}{2(\sigma_k^2)^2} \sum_{i=1}^{n} \gamma_{ik}(x_i - \mu_k)^2 = 0. \tag{25}$$

Solving for σ_k^2:

$$\sigma_k^{2(t+1)} = \frac{\sum_{i=1}^{n} \gamma_{ik}^{(t)} (x_i - \mu_k^{(t+1)})^2}{N_k}. \tag{26}$$

Interpretation: the responsibility-weighted variance around the updated mean. Note we use $\mu_k^{(t+1)}$ (the newly updated mean) in the variance expression.

Summary of M-step updates:

$$N_k = \sum_{i=1}^{n} \gamma_{ik}^{(t)} \tag{27}$$

$$\pi_k^{(t+1)} = \frac{N_k}{n} \tag{28}$$

$$\mu_k^{(t+1)} = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik}^{(t)} x_i \tag{29}$$

$$\sigma_k^{2(t+1)} = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik}^{(t)} (x_i - \mu_k^{(t+1)})^2 \tag{30}$$

4.5 Complete EM algorithm — pseudocode and complexity

Pseudocode (1D / general d only minor adjustments):

1. Initialize $\{\pi_k^{(0)}, \mu_k^{(0)}, \sigma_k^{2(0)}\}_{k=1}^{K}$.
2. For $t = 0, 1, 2, \ldots$ until convergence:
   - **E-step**: For each $i \in \{1, \ldots, n\}$ and $k \in \{1, \ldots, K\}$ compute

$$\gamma_{ik}^{(t)} = \frac{\pi_k^{(t)} \mathcal{N}(x_i \mid \mu_k^{(t)}, \sigma_k^{2(t)})}{\sum_{j=1}^{K} \pi_j^{(t)} \mathcal{N}(x_i \mid \mu_j^{(t)}, \sigma_j^{2(t)})}. \tag{31}$$

   - **M-step**: For each k,

$$N_k = \sum_{i=1}^{n} \gamma_{ik}^{(t)}, \qquad \pi_k^{(t+1)} = \frac{N_k}{n}, \tag{32}$$

$$\mu_k^{(t+1)} = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik}^{(t)} x_i, \qquad \sigma_k^{2(t+1)} = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik}^{(t)} (x_i - \mu_k^{(t+1)})^2. \tag{33}$$

- Compute log-likelihood $\ell(\Theta^{(t+1)})$ and check convergence.

Time complexity:

- E-step dominates: computing K Gaussian densities per data point costs $O(nKd^2)$ in general (for full covariance matrices; in 1D or diagonal covariance it's $O(nK)$).
- M-step is $O(nK)$ for means and variances updates.
- So each EM iteration is $O(nKd^2)$ typically; number of iterations can vary depending on initialization and data.

Convergence criteria:

- Use absolute change in log-likelihood $|\ell^{(t+1)} - \ell^{(t)}| < \epsilon$.
- Or relative change, or a maximum iteration count.

4.6 Initialization strategies and practical tips

Because EM converges to a local maximum, initialization matters. Practical options:

1. **Random initialization**:
   - Randomly pick μ_k from data or from a range, set $\pi_k = 1/K$, and set σ_k^2 to data variance or a fraction thereof.
   - Pros: simple. Cons: may converge to poor local optima.
2. **K-means initialization**:
   - Run K-means to get cluster centers and assign points, then set μ_k to the cluster means, σ_k^2 to the within-cluster variance, and π_k to cluster proportions.
   - Often more robust than random initialization because it gives a reasonable partition.
3. **Multiple restarts**:
   - Run EM with several different initializations and pick the solution with the highest final log-likelihood (or lowest BIC/AIC).
   - This reduces the risk of a poor local maximum.
4. **Hierarchical clustering / domain knowledge**:
   - Use dendrogram cut points or domain-specific heuristics to set initial centers.

Regularization to prevent singularities:

- A pathological case: one component collapses onto a single data point, $\sigma_k^2 \to 0$, causing the likelihood to blow up to infinity because the Gaussian density tends to a delta spike that can explain the point perfectly.
- Practical remedy: impose a minimum variance $\sigma_k^2 \geq \epsilon_{\min}$ (a small positive constant) or add a small regularization term to the covariance (for multivariate case, add λI to the covariance matrix).

5 Theoretical Properties

5.1 Monotonic increase of likelihood — why EM works

High-level idea:

- EM maximizes a lower bound on the observed-data log-likelihood.
- Let $\ell(\Theta) = \log p(X \mid \Theta)$. Fix current parameters $\Theta^{(t)}$.
- Consider the function:

$$\mathcal{F}(q, \Theta) = \mathbb{E}_{q(Z)}\big[\log p(X, Z \mid \Theta)\big] + H(q), \tag{34}$$

  where q is any distribution on the latent variables Z and $H(q)$ is the entropy of q. One can show $\log p(X \mid \Theta) \geq \mathcal{F}(q, \Theta)$ with equality when $q(Z) = p(Z \mid X, \Theta)$.
- EM sets $q(Z) = p(Z \mid X, \Theta^{(t)})$ (E-step), then maximizes $\mathcal{F}$ with respect to Θ (M-step). Because $\mathcal{F}$ is a tight lower bound at the E-step, maximizing it increases the true log-likelihood.
- A formal proof uses Jensen's inequality or Kullback–Leibler divergence.

Consequences:

- EM produces a non-decreasing sequence of log-likelihoods: $\ell(\Theta^{(t+1)}) \geq \ell(\Theta^{(t)})$.
- EM converges to a stationary point but not necessarily the global maximum.

5.2 Identifiability issues and label switching

- **Label switching**: Permuting component indices yields the same mixture model (e.g., swapping labels 1 and 2 does not change the likelihood). Therefore parameters are identifiable only up to a permutation of components.
- This has implications for interpreting component identities across different runs: component k in one run may correspond to component j in another run.

Singularities:
As described earlier, likelihood is unbounded above; without regularization, EM can converge to degenerate solutions.

5.3 Model selection — choosing number of components K

Common criteria:

- **AIC (Akaike Information Criterion)**:

$$\text{AIC} = -2\ell(\hat{\Theta}) + 2p, \tag{35}$$

  where p is the number of free parameters and $\hat{\Theta}$ is the MLE. Smaller AIC indicates a better tradeoff between fit and complexity.
- **BIC (Bayesian Information Criterion)**:

$$\text{BIC} = -2\ell(\hat{\Theta}) + p \log n. \tag{36}$$

  BIC penalizes complexity more strongly than AIC for large n and often leads to smaller models.

- **Cross-validation**:

  Fit GMMs on training folds and evaluate log-likelihood on held-out validation sets (computationally more expensive but principled).

Parameter count p for a GMM in d-dimensional data depends on covariance parametrization (full, diagonal, spherical, tied). For 1D:

Each component contributes 1 mean and 1 variance, plus $K-1$ independent mixing weights (since they sum to 1). So $p = 3K - 1$.

Practical advice:

Try a grid of K values and evaluate BIC/AIC and held-out log-likelihood; examine fits visually when possible.

6 Worked Example: A Simple 1D Gaussian Mixture (expanded)

We will walk through a short, concrete numerical example to make EM operations explicit. This walkthrough focuses on **process and intuition**, not exhaustive arithmetic for each step; however I compute one responsibility fully and show how to proceed for the rest.

Dataset:

$$\mathcal{D} = \{1.0,\ 1.5,\ 2.0,\ 4.0,\ 4.5,\ 5.0\} \tag{37}$$

We assume $K = 2$ components.

6.1 Initialization

Start with:

- $\pi_1^{(0)} = \pi_2^{(0)} = 0.5$
- $\mu_1^{(0)} = 1.5, \quad \mu_2^{(0)} = 4.5$
- $\sigma_1^{(0)} = \sigma_2^{(0)} = 1.0$

Rationale:

Means chosen roughly near the two visible modes; equal mixing weights as a neutral prior; unit variances provide moderate spread.

6.2 E-step: compute responsibilities for each x_i

For each data point x_i, compute:

$$\gamma_{i1} = \frac{0.5 \cdot \mathcal{N}(x_i \mid \mu_1, \sigma_1^2)}{0.5 \cdot \mathcal{N}(x_i \mid \mu_1, \sigma_1^2) + 0.5 \cdot \mathcal{N}(x_i \mid \mu_2, \sigma_2^2)}. \tag{38}$$

Because mixing weights are equal, these responsibilities reduce to normalized likelihood ratios.

Compute one responsibility in full (careful arithmetic for clarity):

For $x_1 = 1.0$, using $\mu_1 = 1.5, \mu_2 = 4.5, \sigma^2 = 1$:

Compute densities:

- For component 1:

$$\mathcal{N}(1.0 \mid 1.5, 1) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(1.0 - 1.5)^2}{2}\right). \tag{39}$$

  Evaluate exponent: $(1.0 - 1.5)^2 = 0.25$, so $-0.25/2 = -0.125$. Then $\exp(-0.125) \approx 0.882497$. The normalizing constant $1/\sqrt{2\pi} \approx 0.398942$. Multiply: $0.398942 \times 0.882497 \approx 0.352065$ (this is a standard value near 0.352).

- For component 2:

$$\mathcal{N}(1.0 \mid 4.5, 1) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{(1.0-4.5)^2}{2}\right). \tag{40}$$

Evaluate exponent: $(1.0-4.5)^2 = 12.25$, so $-12.25/2 = -6.125$. Then $\exp(-6.125) \approx 0.00218$. Multiply by normalizer: $0.398942 \times 0.00218 \approx 0.000869$ — about 8.69×10^{-4}.

Now responsibilities (with equal priors):

$$\gamma_{11} = \frac{0.5 \times 0.352065}{0.5 \times 0.352065 + 0.5 \times 0.000869} = \frac{0.352065}{0.352065 + 0.000869} \approx \frac{0.352065}{0.352934} \approx 0.99753. \tag{41}$$

So $\gamma_{11} \approx 0.998$ (rounded). This indicates component 1 explains x_1 almost entirely.
Interpretation:
Because x_1 is much closer to μ_1 than μ_2, component 1 gets nearly all the responsibility.
Repeat for other points:

- For $x = 1.5$ and $x = 2.0$ responsibilities γ_{i1} will be high (close to 1).
- For $x = 4.0, 4.5, 5.0$ responsibilities γ_{i2} will be high (close to 1).
- Points near the middle of the gap (if present) would have more balanced responsibilities.

Note: you can compute each γ_{ik} similarly. In practice, compute in log-space and normalize to avoid underflow.

6.3 M-step: compute effective counts and update parameters

Compute effective counts:

$$N_1 = \sum_{i=1}^{n} \gamma_{i1}, \quad N_2 = \sum_{i=1}^{n} \gamma_{i2} = n - N_1. \tag{42}$$

Based on the responsibilities pattern above, we expect $N_1 \approx 3$ and $N_2 \approx 3$ because the data clearly splits into two groups of three points each.
Update mixing weights:

$$\pi_k^{(1)} = \frac{N_k}{n}. \tag{43}$$

Update means:

$$\mu_k^{(1)} = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik}\, x_i. \tag{44}$$

Update variances:

$$\sigma_k^{2(1)} = \frac{1}{N_k} \sum_{i=1}^{n} \gamma_{ik}\, (x_i - \mu_k^{(1)})^2. \tag{45}$$

Example of the effect:

- For the left cluster (near 1.0–2.0), the new mean will shift slightly from 1.5 toward the weighted average of 1.0, 1.5, 2.0 (which is exactly 1.5 if responsibilities are exactly 1 for two of those and near 1 for the third). With responsibilities slightly less than 1, μ_1 will move to reflect those weights.
- Variances will likely shrink because the assigned points are close together (so weighted variance is lower than the initial 1.0).

A plausible first iteration (approximate) might yield:

- $\pi_1^{(1)} \approx 0.49,\ \pi_2^{(1)} \approx 0.51$
- $\mu_1^{(1)} \approx 1.8,\ \mu_2^{(1)} \approx 4.6$
- $\sigma_1^{(1)} \approx 0.6,\ \sigma_2^{(1)} \approx 0.7$

(These are illustrative; actual values depend on precise responsibilities.)

6.4 Iterate until convergence

Repeat E and M steps. For small, well-separated datasets like this, EM typically converges in a handful of iterations (often less than 20).

Interpretation of convergence:

- Means stabilize near cluster centers.
- Variances reflect within-cluster spread.
- Mixing weights reflect relative cluster sizes.
- Responsibilities become close to 0 or 1 for well-separated data (hard assignment), but remain fractional near boundaries (soft assignment).

6.5 Visualization suggestions

Plot:

- The data points on the x-axis (e.g., vertical ticks or a histogram).
- The component densities $\pi_k \mathcal{N}(x \mid \mu_k, \sigma_k^2)$ as two curves.
- The mixture density $p(x) = \sum_k \pi_k \mathcal{N}(x \mid \mu_k, \sigma_k^2)$ as the sum curve.
- Optionally, a bar showing γ_{ik} for each point to visualize soft assignments.

7 Summary and practical takeaways

- **GMMs** are a natural probabilistic extension of hard clustering methods like K-means: they provide *soft assignments* (posterior probabilities) and model cluster shape via variances/covariances.
- **EM** is a principled algorithm for maximum-likelihood estimation in models with latent variables. For GMMs, it yields closed-form updates each iteration.
- **Intuitions to remember**:
  - Posterior (γ_{ik}) balances prior belief (π_k) and explanatory power of a component (likelihood).
  - M-step updates are just weighted MLEs where weights are the responsibilities.
  - EM raises the log-likelihood monotonically but may reach a local maximum; initialization matters.
- **Common pitfalls and fixes**:

  - Numerical underflow in E-step → compute in log-space.
  - Variance collapse (singularity) → enforce minimum variance or regularize covariance.
  - Label switching → be mindful when comparing components across runs.

- **Model selection**: Use BIC or AIC to trade off fit vs complexity. Visual inspection and domain knowledge also help.

8 Appendix: Additional mathematical and practical remarks

8.1 Posterior vs likelihood vs prior — plain-language definitions

- **Prior**: before seeing data, how probable is a hypothesis. In GMMs, π_k is the prior probability that a data point belongs to component k.
- **Likelihood**: given a component and its parameters, how probable is the observed data. For point x and component k, the likelihood is $\mathcal{N}(x \mid \mu_k, \sigma_k^2)$.
- **Posterior**: after seeing the data, how probable is the hypothesis. Posterior $p(z = k \mid x)$ combines prior and likelihood via Bayes' theorem: posterior $\propto$ prior $\times$ likelihood.
- **Intuition**: prior is your baseline guess, the likelihood is how well a hypothesis explains the observed data, and the posterior is the updated belief mixing both.

8.2 Log-sum-exp trick (practical formula)

To compute $\log\left(\sum_j e^{a_j}\right)$ stably:

- Let $m = \max_j a_j$.
- Then $\log\left(\sum_j e^{a_j}\right) = m + \log\left(\sum_j e^{a_j - m}\right)$. This prevents exponentiating very negative numbers which would underflow.

8.3 Covariance types in multivariate GMMs (brief)

- **Full**: each component has its own full covariance matrix (flexible, many parameters).
- **Diagonal**: covariance matrix is diagonal (assumes independence across dimensions).
- **Spherical**: covariance is scalar times identity ($\sigma^2 I$).
- **Tied**: all components share the same covariance matrix. Choice affects flexibility, number of parameters, and risk of overfitting.

8.4 Implementation checklist for practitioners

- Standardize/scale features before fitting GMMs.
- Use K-means initialization for robustness.
- Try several covariance types and number of components.
- Compute BIC/AIC and check held-out likelihood.
- Visualize fits when possible.